```
template<typename Iterator,typename Func>
void parallel_for_each(Iterator first,Iterator last,Func f)
{
    unsigned long const length=std::distance(first,last);

    if(!length)
        return;

    unsigned long const min_per_thread=25;

    if(length<(2*min_per_thread))
    {
        std::for_each(first,last,f);      <-1
    }
    else
    {
        Iterator const mid_point=first+length/2;
        std::future<void> first_half=                    <-2
            std::async(&parallel_for_each<Iterator,Func>,
                       first,mid_point,f);
        parallel_for_each(mid_point,last,f);     <-3
        first_half.get();                <-4
    }
}
```

As with your `std::async`-based `parallel_accumulate` from listing 8.5, you split the data recursively rather than before execution, because you don't know how many threads the library will use. As before, you divide the data in half at each stage, running one half asynchronously ❷ and the other directly ❸ until the remaining data is too small to be worth dividing, in which case you defer to `std::for_each` ❶. Again, the use of `std::async` and the `get()` member function of `std::future` ❹ provides the exception propagation semantics.

Let's move on from algorithms that must perform the same operation on each element (of which there are several; `std::count` and `std::replace` spring to mind for starters) to a slightly more complicated example in the shape of `std::find`.

8.5.2. A parallel implementation of std::find